

Conditioned LoRA in Bolero

Technical model note

July 2026

Abstract

Bolero is a cell-state-conditioned sequence-to-function model. It combines a frozen Borzoi/Flashzoi DNA backbone with low-rank adapters whose weights are generated from a cell-state embedding. The resulting effective network is different for each conditioning vector while sharing the expensive sequence model across cells. This note describes the conditioning path, the conditional LoRA parameterization, the paired signal-model training recipe, and inference by collapsing the adapters into a fixed network.

1 Model overview

The model takes a long DNA sequence and a representation of the target cell state. The default sequence input is one-hot DNA of length 524,288 bp. The output is a chromatin or transcript track at 32 bp resolution. The backbone produces 16,384 internal output bins; the default training crop is 16,352 bins (see `crop_to_length` in `src/bolero/tl/model/borzoi/model.py`).

The main implementation is `BorzoiLoRA` in `src/bolero/tl/model/borzoi/model_lora.py`. The base checkpoint is loaded first, and selected convolutional and linear layers are then replaced by LoRA modules.

2 Conditional LoRA

2.1 From ordinary LoRA to conditional LoRA

For a linear layer with frozen weight $W_0 \in \mathbb{R}^{o \times i}$, ordinary LoRA uses a learned, input-independent low-rank update. Bolero instead generates both low-rank factors from the conditioning vector:

$$A(e) \in \mathbb{R}^{r \times i}, \tag{1}$$

$$B(e) \in \mathbb{R}^{o \times r}, \tag{2}$$

$$W(e) = W_0 + \frac{\alpha}{r} B(e) A(e). \tag{3}$$

Each factor is produced by a small MLP, implemented as an `EmbeddingMLP`. The MLPs are shared across all examples; only their outputs vary with e . This amortizes the cost of learning a separate effective network for every cell state.

The implementation stores the first factor with shape (r, i) and the second with shape (o, r) , contracts them over the rank dimension, and adds the result to the frozen base weight. The scale is α/r ; with the default configuration, α is obtained from the rank and the configured LoRA scale.

2.2 Explicit convolutional parameterization

For a one-dimensional convolution with base kernel

$$K_0 \in \mathbb{R}^{o \times (i/g) \times k},$$

where i and o are the input and output channel counts, g is the number of groups, and k is the kernel width, the implementation flattens the channel–kernel dimensions before applying the low-rank factorization. For conditioning vector e , its two MLPs generate

$$A_c(e) \in \mathbb{R}^{(rk) \times ((i/g)k)}, \quad (4)$$

$$B_c(e) \in \mathbb{R}^{o \times (rk)}. \quad (5)$$

The generated convolution update is

$$\Delta K(e) = \frac{\alpha}{r} \text{reshape}_{o, i/g, k} [B_c(e) A_c(e)],$$

and the effective kernel for one example is

$$K(e) = K_0 + \Delta K(e).$$

Thus, the factorization rank in the flattened representation is rk ; the code’s LoRA rank parameter is r , and the kernel width is folded into both generated factors. For a two-dimensional convolution, the analogous generator shapes are

$$A_c(e) \in \mathbb{R}^{(rk) \times ((i/g)k)}, \quad B_c(e) \in \mathbb{R}^{(ok) \times (rk)},$$

followed by reshaping $B_c A_c$ to $(o, i/g, k, k)$.

For a batch of b examples, the one-dimensional implementation generates $K(e_1), \dots, K(e_b)$, concatenates them into a grouped-convolution weight tensor

$$K_{\text{batch}} \in \mathbb{R}^{(bo) \times (i/g) \times k},$$

reshapes the input $X \in \mathbb{R}^{b \times i \times L}$ to $X' \in \mathbb{R}^{1 \times (bi) \times L}$, and calls convolution with bg groups. The output is reshaped back to $Y \in \mathbb{R}^{b \times o \times L'}$. Each example therefore uses its own conditional kernel while the batch is processed in one grouped operation.

The conditional layer implementation is in `src/bolero/tl/generic/module_lora_cond.py`. Its two defining properties are:

1. the pretrained base weight is frozen;
2. the A and B generators are trainable and initialized so the initial LoRA update is zero (the final layer of the B generator is zero-initialized).

Zero initialization makes the converted model initially behave like the pretrained backbone, before the adapters learn a cell-state-dependent correction.

2.3 Which layers are adapted?

The default preset is `all_conditional`. Its principal ranks are:

Backbone region	LoRA rank
DNA convolution	2
Residual tower and first U-Net block	6
Transformer-associated layers	30
Horizontal and upsampling convolutions	30
Separable convolutions	10
Final joined convolutions	60
Output heads	1

Most selected layers use conditional LoRA. The transformer query, key, value, relative-key, and output projections are explicitly excluded from conditional LoRA; they receive ordinary input-independent LoRA adapters instead. This exception is specified by `exclude_cond_lora_patterns` in `src/bolero/tl/model/borzoi/model_lora_config.py`.

3 Building the conditioning vector

3.1 What is the cell-state embedding?

In the current embedding tutorial, the cell-state embedding is *not* a set of expression principal components. It is a learned latent representation from a joint RNA-ATAC MULTIVI model (implemented by `scvi-tools`). The worked tutorial uses a 30-dimensional latent vector. MultiVI is trained on raw RNA counts for the top 5,000 highly variable genes and ATAC peak counts, with donor/experimental batch modeled as a covariate. The trained model is then applied to all cells, and the latent representation is stored as `X_multivi / multivi.latent_embedding.feather`.

SEACells uses this joint latent space to form homogeneous metacells. Bolero does not usually feed individual noisy cells directly to the sequence model: the pseudobulk records carry the embedding associated with each metacell or pseudobulk. A pseudobulk embedding is typically the mean of the embeddings of its selected metacells. Therefore, the embedding is a learned multimodal cell-state coordinate, not an expression-only PCA score. The embedding dimensionality and model can be changed by the data-preparation workflow; the LoRA input width is read from the resulting records. The LoRA code itself is agnostic to the embedding’s provenance: an expression-PCA vector could be supplied through the same `emb_key` interface, but that is not the embedding used by the current MULTIVI-based tutorial and atlas recipe.

3.2 Single-dataset training

The paired pseudobulk pipeline samples a background pseudobulk p_0 and a target pseudobulk p_1 . Their cell-state embeddings are concatenated:

$$e_{\text{cell}} = [e_{p_0}; e_{p_1}].$$

For example, two 30-dimensional metacell embeddings give a 60-dimensional input to the LoRA generators. The background embedding describes the reference state, while the target embedding describes the state whose track is to be predicted. The construction is performed by `EnsemblePairedPseudobulker` in `src/bolero/tl/pseudobulk/paired_pseudobulk.py`.

Optional condition variables, such as tissue, developmental state, or TF activity, are encoded separately. `ConditionEmbeddingModule` applies small encoders to the cell and condition inputs and concatenates their outputs. If no condition variables are present, the module is replaced by a no-effect module and the cell embedding is passed through unchanged.

3.3 Multi-dataset training

Different datasets can have different embedding spaces and condition schemas. The multi-dataset model therefore uses:

- a dataset-specific cell encoder;
- a dataset-specific condition encoder when conditions exist;
- a shared encoder for genome/species and other shared metadata.

The resulting dataset-specific and shared representations are concatenated into one vector of fixed width before being supplied to all conditional-LoRA generators. This allows one shared backbone to serve multiple datasets while retaining dataset-specific conditioning.

4 Signal-model forward pass

With `signal_model=True`, the model receives the background signal from p_0 in addition to DNA. The signal is transformed with $\log(1 + s)$ and passed through a small convolutional encoder. Its output is injected into the DNA representation at 32 bp resolution. The subsequent Borzoi residual tower, U-Net path, transformer, and upsampling path all receive the same conditioning vector and therefore use state-specific adapters.

The target output is produced by a 1D output head. For the default count head, a softplus activation makes predictions nonnegative and training uses the Poisson-multinomial loss. In paired training, the model can also predict the difference between target and background signals with an auxiliary delta head and mean-squared-error loss.

At training time, the background signal is randomly omitted with probability `nosignal_prob` (default 0.2). This prevents the model from depending entirely on the background track and encourages prediction from DNA plus conditioning. At inference time, the signal input may be omitted; the DNA and conditioning paths remain available.

5 Training and trainable parameters

The trainer first loads the pretrained Borzoi checkpoint and freezes the backbone. It then converts the configured layers to LoRA modules. The base convolution and linear weights remain frozen, while the following components are trainable in the standard recipe:

- conditional-LoRA generator MLPs;
- ordinary LoRA parameters in the attention projections;
- the cell/condition encoders;
- the output heads;

- the signal encoder and enabled auxiliary heads.

The optimizer is built from parameters whose `requires_grad` flag is true. The backbone is thus reused as a strong sequence prior, while the comparatively small adapter and conditioning components learn cell-state-specific regulation.

6 Collapsing for inference

For a fixed conditioning vector e^* , every conditional layer can be evaluated once:

$$W^* = W_0 + \frac{\alpha}{r} B(e^*) A(e^*).$$

`BorzoiLoRA.collapse_lora(embedding)` replaces the conditional layers with ordinary PyTorch layers containing these fixed weights. The result is a conventional DNA-to-track model for one cell state. It is useful for fast repeated prediction and for base-level DNA attribution. A different collapsed model is needed for each conditioning vector whose effective network is to be inspected.

7 Implementation map

Component	Source file
Conditional linear and convolutional LoRA	<code>src/bolero/tl/generic/module_lora_cond.py</code>
Condition encoders and MLP generators	<code>src/bolero/tl/generic/module_embedding.py</code>
BorzoiLoRA construction and forward paths	<code>src/bolero/tl/model/borzoi/model_lora.py</code>
Layer/rank presets	<code>model_lora_config.py</code>
Paired training forward pass	<code>src/bolero/tl/model/borzoi/train.py</code>
Pseudobulk pairing and embeddings	<code>src/bolero/tl/pseudobulk/paired_pseudobulk.py</code>